Listing 3.4 A message decoder function that extracts temperature and humidity from the payload of a Dragino LHT52 sensor.

```
def decode_sensor_message(base64_payload):
  raw_bytes = base64.b64decode(base64_payload) #1

  temp_raw = struct.unpack('>h', raw_bytes[0:2])[0] #2
  temperature = temp_raw / 100.0 #3

  humidity_raw =
    struct.unpack('>H', raw_bytes[2:4])[0] #4
  humidity = humidity_raw / 10.0 #5

  return {
    'temperature': temperature,
    'humidity': humidity,
    'unit': 'celsius'
  }

payload = "B1QDOH//AWhRYAQ=" #6
data = decode_sensor_message(payload)
print(f"Temperature: {data['temperature']}°C")
print(f"Humidity: {data['humidity']}%")
```

#1 Convert Base64 to raw bytes.
#2 Extract temperature (bytes 0-1): signed 16-bit, big-endian.
#3 Convert to degrees Celcius.
#4 Extract humidity (bytes 2-3): unsigned 16-bit, big-endian.
#5 Convert to percentage.
#6 Example encoded payload from sensor.

This transformation serves multiple purposes:

- *Decoding*: converts binary data to human-readable values.
- *Standardization*: creates consistent JSON structure across sensor types.
- *Context addition*: adds units and metadata not present in raw data.

3.5.3 Sensor fusion

Real world sensor data is often noisy and uncertain. Each sensor provides an imperfect view of the true state of the system, and in a digital twin, relying on a single sensor can lead to an inaccurate representation of the physical system.